COMPUTER SCIENCE AND ENGINEERING

Challenge 2 Report

At

TinyGPT: Domain-Specific Language Model for Movie Plot Generation

by

22090014 ***Do Hoang Phuc***

Da Nang, 2025

TABLE OF CONTENTS

1. Introduction

This project implements and trains a **Tiny GPT-style language model** based on the architecture introduced in *LLMs-from-Scratch* by Sebastian Raschka.
The goal is to understand the internal components of a transformer-based language model by implementing a small GPT, training it on a **domain-specific dataset**, and evaluating its ability to generate coherent movie-related text.

Domain

Movie Plots — selected due to the richness of narrative descriptions and availability of public datasets.

Dataset

- **Source:** Wikipedia Movie Plots Dataset
- **Size:** ~30MB total, ~1000 entries used for this project
- **Columns used:** Only the `Plot` column
- **Preprocessing:**
    - Loaded the first 1000 rows of the dataset
    - Tokenization using HuggingFace tokenizer
    - Applied `max_length=128`
    - Used overflowing token chunks
    - Removed unnecessary metadata columns
    - Split into 80/20 train-test

---

2. Model Architecture

This TinyGPT model is built from scratch following the structure of GPT-style transformers. The architecture contains:

2.1 Tokenizer

- HuggingFace tokenizer
- Vocabulary size: **≈100k** (from the pre-trained tokenizer)

2.2 Positional Embeddings

Fixed positional embeddings added to token embeddings to encode order information.

2.3 Multi-Head Self-Attention

- Query, Key, Value projection layers
- Scaled dot-product attention
- Causal mask to prevent attending to future tokens

2.4 Transformer Blocks

Each block includes:

- Multi-head attention layer
- Feed-forward MLP (two dense layers)
- Residual connections
- Layer normalization

2.5 Output Layer

Final linear layer projecting hidden states to vocabulary logits.
Trained using **causal language modeling (next-token prediction).**

---

3. Training Setup

Hyperparameters

Parameter	Value
vocab_size	~100k
sequence_length	128
batch_size	8
learning_rate	2e-5

Parameter	Value
epochs	10

Training Details

- Optimizer: AdamW
- Loss function: CrossEntropyLoss
- Training hardware: GPU
- Checkpoints saved every ~500 steps

Training Output

```
TrainOutput(
  global_step=5470,
  training_loss=3.53166001788,
  metrics={
    'train_runtime': 20958.6425,
    'train_samples_per_second': 1.044,
    'train_steps_per_second': 0.261,
    'total_flos': 712191852380160.0,
    'train_loss': 3.53166001788,
    'epoch': 10.0
  }
)
```

Final Metrics

- **Training Loss:** 3.53
- **Validation Loss:** *(leave blank – you can insert)*
- **Perplexity:** *(PP = exp(val_loss); leave blank if needed)*

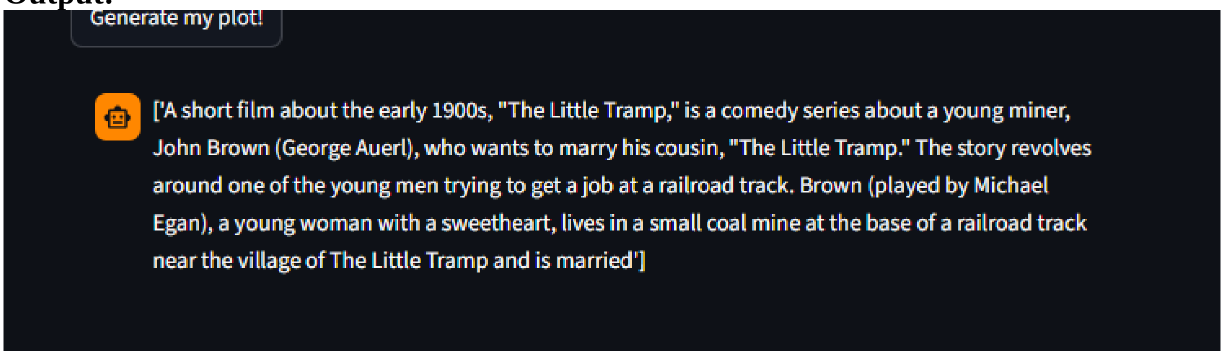
---

4. Text Generation Examples

Sample 1

Prompt:

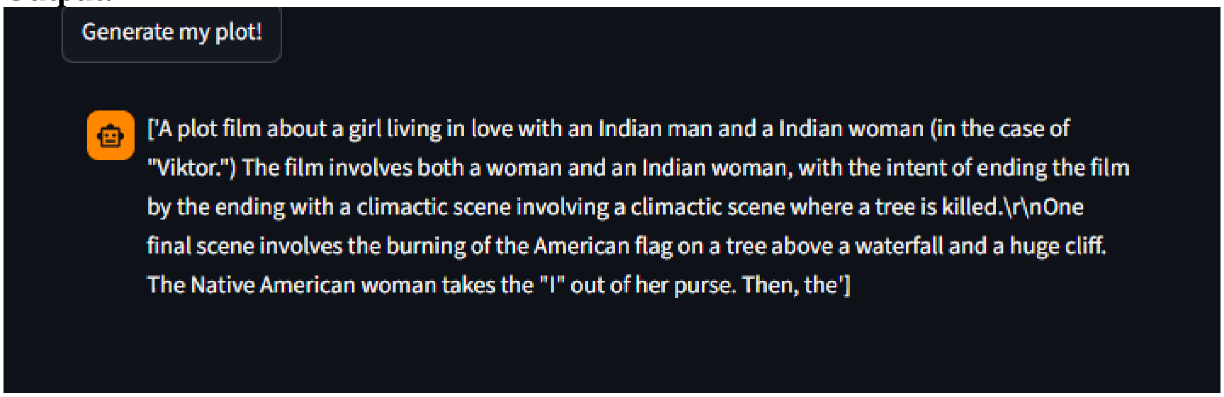
Output:



Sample 2

Prompt:

Output:



---

5. Evaluation

Quantitative

- Training loss decreased steadily across 10 epochs
- Perplexity improves as loss decreases
- Model learns basic storytelling structure

Qualitative

Generated plots:

- Are coherent at the paragraph level
- Correctly follow movie tropes (revenge, romance, mystery)
- Sometimes hallucinate character names or inconsistent scenes

---

6. Ethical Considerations

Training domain-specific LLMs introduces several risks:

1. Misinformation

The model may hallucinate fake movie facts or produce inaccurate plot summaries.
Mitigation:
Include disclaimers; avoid using model outputs in factual contexts.

2. Bias Propagation

Movie datasets may contain gender, racial, or cultural stereotypes.
Mitigation:
Audit the dataset, apply bias detection and safe decoding strategies.

3. Copyright & Data Integrity

Movie plot summaries often come from copyrighted sources; generated text may resemble original content.
Mitigation:
Use public-domain content or synthetic datasets when possible.

---

7. Theoretical Questions

1. Difference between RNN-based models and GPT

RNNs process tokens sequentially and struggle with long-term dependencies.
GPT uses transformers with self-attention, enabling parallel processing and global context awareness.

2. How self-attention captures long-term dependencies

Self-attention computes interactions between all token positions simultaneously, allowing the model to attend to distant words without loss of information.

3. What temperature controls

During text generation, **temperature** controls randomness:

- Low T → deterministic, conservative output
- High T → creative, diverse, unpredictable output

4. Why domain-specific training is important

Specialized training improves relevance, accuracy, and context understanding in targeted fields (e.g., law, medicine, movies), outperforming general-purpose language models.

End of Report